

VoiceLab — Master Build Documentation

Channel: Phase Alpha
Project: VoiceLab (AI Voice Platform)
Started: March 2026
Purpose: Step-by-step build log for development reference and YouTube video planning

Project Overview

VoiceLab is a full-stack AI voice generation platform built on a student budget with no local GPU. It uses Kokoro TTS (open source, Apache 2.0) for text-to-speech generation. Series 2 will extend the same platform with OpenVoice V2 for voice cloning.

Tech Stack

Layer	Tool	Cost
Frontend	React.js	Free (Vercel)
Backend	FastAPI (Python)	Free (Render)
Database	Supabase (PostgreSQL)	Free tier
TTS Model	Kokoro TTS	Free (Apache 2.0)
GPU Inference	RunPod Serverless	~\$10 minimum load
Dev/Testing GPU	Kaggle	Free (30hr/week)
Local Dev	Laptop CPU	Free
Containerization	Docker	Free
Version Control	Git + GitHub	Free

Architecture Summary



↓
React Frontend → Vercel
↓
FastAPI Backend → Render
↓
RunPod Serverless Endpoint (Kokoro TTS Docker Image)
↓
Supabase (stores job status + audio URL)
↓
Frontend polls → receives audio → plays to user

Why webhook/polling architecture?

Render free tier times out at 30 seconds. Kokoro TTS inference takes 10-60 seconds.

Solution: Backend dispatches job to RunPod → returns job_id immediately → RunPod calls back when done → frontend polls job status.

Model Choices & Licenses

Model	License	Use Case
Kokoro TTS	Apache 2.0 	Series 1 — Text to Speech
OpenVoice V2	MIT 	Series 2 — Voice Cloning

Both are fully free including commercial use. No legal issues.

Development Workflow

Write code (Laptop CPU)
→ Test API/DB/Auth locally
→ Test model quality (Kaggle T4 GPU)
→ Build Docker image
→ Deploy to RunPod
→ Deploy backend to Render
→ Deploy frontend to Vercel

Series Roadmap

Video	Title	Status
1	What we're building — big picture + architecture	🕒 Script in progress
2	Project structure + environment setup	❌ Not started
3	Core FastAPI backend locally	❌ Not started
4	Kokoro TTS integration + local testing	❌ Not started
5	Docker setup + RunPod serverless deployment	❌ Not started
6	React frontend build	❌ Not started
7	Deploying backend (Render) + frontend (Vercel)	❌ Not started
8	Webhook architecture — why and how	❌ Not started
9	Auth + database + real user flows	❌ Not started
10	Production debugging — real problems, real fixes	❌ Not started

PHASE 1 — SYSTEM SETUP

System Info

- **OS:** Windows (new laptop, clean install)
- **RAM:** 32GB
- **Storage:** 1TB
- **GPU:** None (using Kaggle for GPU testing)
- **Date started:** March 2026

Day 1 System State

```
python --version → Python 3.14.3 (pre-installed, too new for ML)
node --version   → v24.14.0 ✅
git --version    → git version 2.53.0.windows.2 ✅
docker --version → Docker version 29.2.1 ✅
```

Important note on Python versions:

Laptop had Python 3.14.3 pre-installed. Python 3.14 is too new — PyTorch and ML libraries don't

support it yet.

Installed Python 3.11.9 separately. Both versions coexist on the system.

Virtual environment uses 3.11.9 specifically via `py -3.11` command.

STEP 1 — Install Python 3.11

Why Python 3.11?

Stable, widely supported, compatible with FastAPI, Kokoro TTS, and all dependencies.

Avoid Python 3.12+ for now — some ML libraries have compatibility issues.

Download:

→ <https://www.python.org/downloads/release/python-3119/>

→ Scroll down → Find "Windows installer (64-bit)" → Download

Installation settings — CRITICAL:

☒ Check "Add Python to PATH" — this is the most important checkbox

☒ Check "Install pip"

→ Click "Install Now"

Verify installation — run in terminal:

```
bash
python --version
pip --version
```

Actual output:

```
Python 3.11.9
pip 26.0.1
```

Check all installed Python versions:

```
bash
py --list
```

Output:

```
-V:3.14 *      Python 3.14.3
-V:3.11        Python 3.11 (64-bit)
```

Status:  Installed

STEP 2 — Install Node.js

Why Node.js?

Required for React frontend development and npm package management.

Actual versions installed:

```
node --version → v24.14.0
npm --version → 11.9.0
```

Status:  Already installed

STEP 3 — Install Git

Why Git?

Version control for all our code. Required for GitHub, deployment to Render/Vercel, and Docker builds.

Actual version:

```
git version 2.53.0.windows.2
```

Status:  Already installed

STEP 4 — Install VS Code

Why VS Code?

Best free code editor. Extensions for Python, React, Docker all available.

Extensions installed:

-  Python (by Microsoft)
-  Prettier — Code formatter
-  ES7+ React/Redux/React-Native snippets
-  Docker (by Microsoft)
-  GitLens (by GitKraken)

Python Interpreter configured:

→ Selected `Python 3.11.9 (venv)` → `.\venv\Scripts\python.exe` → Marked as

Recommended

Status:  Installed and configured

STEP 5 — Install Docker Desktop

Why Docker?

We need Docker to containerize our Kokoro TTS model for RunPod deployment. This is non-negotiable — RunPod only accepts Docker images.

Actual version:

```
Docker version 29.2.1, build a5c7197
```

Note: Docker Desktop needs to be running (open) whenever you use Docker commands.

Status:  Already installed

STEP 6 — Fix PowerShell Execution Policy

Problem encountered:

Running `.\venv\Scripts\activate` gave `SecurityError` — PowerShell blocks scripts by default on Windows.

Solution:

```
bash

Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

What this does:

- `RemoteSigned` → local scripts run freely, downloaded scripts must be signed
- `CurrentUser` → only affects your user account, not the entire system
- Safest fix — not opening up everything, just enough for project scripts

Status:  Fixed

STEP 7 — Create Project Folder Structure

Location: `D:\Projects\voicelab`

Why D drive?

Keeps project files separate from system files on C drive. Cleaner organization.

Commands run:

```
bash
cd D:\
mkdir Projects
cd Projects
mkdir voicelab
cd voicelab
```

Create main service folders:

```
bash
mkdir backend
mkdir frontend
mkdir runpod
```

Current folder state:

```
D:\Projects\voicelab\
├─ backend\
├─ frontend\
├─ runpod\
└─ venv\
```

Status:  Done

STEP 8 — Create Virtual Environment

Why virtual environment?

Each project gets its own isolated room. Libraries installed for VoiceLab stay inside VoiceLab. Prevents version conflicts between different projects on the same machine.

Command — using Python 3.11 specifically:

```
bash
py -3.11 -m venv venv
```

Activate:

```
bash

.\venv\Scripts\activate
```

Verify correct Python inside venv:

```
bash

python --version
→ Python 3.11.9 ✓
```

Upgrade pip inside venv:

```
bash

python -m pip install --upgrade pip
→ Successfully installed pip-26.0.1 ✓
```

Daily habit — every time you open VS Code:

1. Open VS Code
2. Open terminal (Ctrl + `)
3. Check if (venv) is showing
4. If not → run .\venv\Scripts\activate
5. Then start working

Status: ✓ Done

STEP 9 — Create GitHub Account (Phase Alpha) ✓

GitHub username: phasealpha-ai

Email: sphasealpha@gmail.com

Repo: <https://github.com/phasealpha-ai/voicelab>

License: MIT

Visibility: Public

Git configured locally:

```
bash

git config --global user.name "Phase Alpha"
git config --global user.email "sphasealpha@gmail.com"
```


Authentication: Personal Access Token (classic) with repo scope

→ GitHub stopped accepting passwords in 2021 — PAT required for push access

Problem encountered:

Git opened Vim editor for merge commit message — confusing for first time users.

Solution: Use `--no-edit` flag to skip Vim entirely:

```
bash
git commit --no-edit
```

Problem encountered:

403 Permission denied on first push attempt.

Solution: Generate Personal Access Token (classic) from GitHub Settings → Developer Settings
→ Tokens (classic) → repo scope

First push successful:

```
7 objects pushed
branch 'main' set up to track 'origin/main'
```

Status:  Done

STEP 10 — Initialize Git Repository

Commands run:

```
bash
git init
New-Item .gitignore -ItemType File
New-Item README.md -ItemType File
New-Item .env.example -ItemType File
git add .
git commit -m "initial commit: project structure and gitignore"
git remote add origin https://github.com/phasealpha-ai/voicelab.git
git branch -M main
git pull origin main --allow-unrelated-histories
git commit --no-edit
git push -u origin main
```

Files in repo:

- ✓ .gitignore → protects venv, .env, node_modules, __pycache__
- ✓ .env.example → safe template for environment variables
- ✓ README.md → placeholder, full README after project complete
- ✓ LICENSE → MIT License

Status: ✓ Done

STEP 11 — Create Online Service Accounts ✖

All free. Create these using Phase Alpha email.

Service	URL	Purpose	Status
GitHub	github.com	Code hosting + version control	✓ Done
Supabase	supabase.com	Database + auth + storage	✖
RunPod	runpod.io	Serverless GPU inference	✖
Render	render.com	Backend hosting	✖
Vercel	vercel.com	Frontend hosting	✖
Kaggle	kaggle.com	Free GPU for testing	✖
Docker Hub	hub.docker.com	Store Docker images	✖

PHASE 2 — PROJECT STRUCTURE SETUP

Complete Folder Structure Plan

```
D:\Projects\voicelab\
|
├─ backend\           → FastAPI application (deploys to Render)
|   └─ app\
|       └─ routes\     → API endpoints (tts.py, auth.py, jobs.py)
|       └─ services\   → business logic (runpod_service.py,
supabase_service.py)
|       └─ models\     → database models / pydantic schemas
|       └─ main.py      → FastAPI entry point
```

— Dockerfile	→ for Render deployment
— requirements.txt	→ backend Python dependencies
— .env	→ backend secrets (NEVER pushed to GitHub)
— frontend\	→ React application (deploys to Vercel)
— src\	
— components\	→ reusable UI components
— pages\	→ individual pages (Home, Dashboard, Auth)
— services\	→ API call functions (api.js)
— App.jsx	→ root component
— public\	
— .env	→ frontend secrets (NEVER pushed to GitHub)
— runpod\	→ GPU inference handler (deploys to RunPod)
— handler.py	→ RunPod serverless entry point
— Dockerfile	→ for RunPod deployment
— requirements.txt	→ model-specific dependencies (Kokoro TTS)
— .gitignore	→ tells Git what NOT to push (venv, .env,
— __pycache__)	
— .env.example	→ safe template showing variable names without real
values	
— README.md	→ project documentation

Why this structure is industry level:

- Each service (backend, frontend, runpod) is independently deployable
- No service knows about another's internals — they communicate only via APIs
- Secrets never leave the machine via .gitignore
- Anyone can clone the repo and know exactly what env variables they need via .env.example
- This is called a **monorepo with separated services** architecture

Status: 🕒 Folders created, files not yet created

PHASE 3 — BACKEND DEVELOPMENT

(Will be filled after Phase 2 is complete)

PHASE 4 — MODEL INTEGRATION

(Will be filled after Phase 3 is complete)

PHASE 5 — DOCKER + RUNPOD DEPLOYMENT

(Will be filled after Phase 4 is complete)

PHASE 6 — FRONTEND DEVELOPMENT

(Will be filled after Phase 5 is complete)

PHASE 7 — FULL DEPLOYMENT

(Will be filled after Phase 6 is complete)

Problems & Solutions Log

(Every bug, error, and fix gets documented here as we go)

#	Problem	Solution	Phase
—	None yet	—	—

Key Decisions Log

Decision	Choice	Reason
TTS Model	Kokoro TTS	Apache 2.0, free, lightweight, CPU-capable
Voice Cloning Model	OpenVoice V2	MIT license, zero-shot cloning, actively maintained
Backend Framework	FastAPI	Fast, async, Python-native, industry standard
Frontend Framework	React.js	Industry standard, component-based, Vercel-optimized
Database	Supabase	Free tier generous, auth built-in, PostgreSQL

Decision	Choice	Reason
GPU Platform	RunPod Serverless	Pay per second, no idle cost, Docker-based
Dev GPU	Kaggle	Free 30hr/week T4 GPU
Architecture	Webhook + polling	Render 30s timeout + AI inference 10-90s mismatch
Channel Name	Phase Alpha	Reflects student journey, raw and real
Project Name	VoiceLab	Clean, memorable, searchable

This document is updated after every build session.
Last updated: March 2026 — Phase 1 complete  / Phase 2 starting